

LABORATORY

LAB 07

Tone Fundamentals

Generating Sound with a Passive Buzzer using EduFramework

S32K144 · MaaZEDU

EduFramework

FPT University

Contents

1	Introduction	2
1.1	Overview	2
1.2	Objectives	2
2	Background	2
2.1	Passive Buzzer and Control Signal	2
2.2	Period and Frequency	2
2.3	Frequency and Pitch	3
2.4	Frequency and Tone Duration	3
3	Hardware Setup	3
3.1	Required Hardware	3
3.2	Passive Buzzer Connection	3
4	EduFramework API	4
4.1	<code>tone()</code>	4
4.2	<code>noTone()</code>	5
4.3	PWM Resource Note	5
5	Lab Exercise	5
5.1	Requirements	5
5.2	Program	6
5.3	Verification	7
6	Extension	7
6.1	Naming Frequencies with Constants	7
6.2	Extension Exercise: Create a Short Melody	7
7	References	7

1 Introduction

1.1 Overview

Sound is commonly used in embedded systems to indicate status, issue alerts, or provide feedback to the user. With a passive buzzer, sound can be generated using a periodic signal; changing the signal frequency changes the perceived pitch [2] [3] .

This lab introduces the generation of simple tones using EduFramework. A passive buzzer is connected to `GPI07` , and the program sequentially generates different frequencies to produce a series of tones with increasing pitch. The lab focuses on three main concepts: periodic signals, the relationship between frequency and pitch, and buzzer control using the framework API.

1.2 Objectives

OBJECTIVES

The objectives of this laboratory are:

1. Explain the basic principle of generating sound with a passive buzzer.
2. Describe the relationship among period, frequency, and sound pitch.
3. Use `tone()` and `noTone()` to control tones with EduFramework.
4. Build and verify a program that plays a sequence of tones on a passive buzzer.

2 Background

2.1 Passive Buzzer and Control Signal

A passive buzzer does not generate a fixed tone when driven by a constant logic level. To produce sound, the buzzer must be excited by a signal that changes periodically over time. In embedded-system exercises, a square wave is a common way to generate this excitation signal [2] [3] .

A square wave alternates between two logic levels. When these transitions repeat at a regular rate, the signal has a defined frequency. By changing the control frequency, the same buzzer can produce tones with different pitches [2] .

Within the scope of this lab, the buzzer is treated as a simple sound-output device. More advanced characteristics such as frequency response, sound pressure level, or the electrical model of the buzzer element are not considered.

2.2 Period and Frequency

A periodic signal repeats after a defined interval of time. The time required to complete one repetition is called the period and is commonly denoted by T . Frequency indicates the number of cycles that occur in one second and is expressed in hertz (Hz) [1] .

For a periodic signal, frequency and period are inversely related:

$$f = \frac{1}{T}$$

where:

- f is the frequency, measured in hertz.

- T is the period, measured in seconds.

Therefore, a higher frequency corresponds to a shorter period. For example, a 500 Hz signal repeats faster than a 250 Hz signal, so the time between cycles of the 500 Hz signal is shorter.

2.3 Frequency and Pitch

Pitch is the perception of how high or low a sound is. For simple periodic tones, pitch is closely related to frequency: increasing the frequency raises the perceived pitch, while decreasing the frequency lowers it [1]. This relationship forms the basis of the exercise. When the buzzer receives a sequence of signals with increasing frequencies, the resulting tones are also heard as progressively higher in pitch [2].

Frequency	Period	Perceived Pitch
Lower	Longer	Lower-pitched sound.
Higher	Shorter	Higher-pitched sound.

Table 7.2.1. Qualitative relationship among frequency, period, and pitch

2.4 Frequency and Tone Duration

Frequency determines the pitch of a tone, while duration determines how long the tone is maintained. These quantities serve different purposes: a tone can keep the same frequency while being played for a shorter or longer interval depending on the application. Buzzer exercises commonly describe frequency in hertz and duration separately in milliseconds [3].

In Lab 07, each tone is maintained for 500 ms. After the tone sequence, the buzzer is stopped for 1000 ms to create a silent interval before the sequence starts again.

3 Hardware Setup

3.1 Required Hardware

This lab uses a MaaZEDU Development Board based on the S32K144 microcontroller and an external passive buzzer [7].

Hardware	Description
MaaZEDU Development Board	Development board based on the S32K144 microcontroller.
Passive Buzzer	Sound-output device driven by a periodic signal.
Jumper wires	Connect the buzzer to the board.
USB Cable	Connect the board to the computer for power and programming.

Table 7.3.1. Hardware required for the lab

3.2 Passive Buzzer Connection

GPIO7 is a PWM-capable Logical Pin in EduFramework and is used as the buzzer control output in this lab [6].

Device	Connection	Function
Passive Buzzer	GPIO7	Receives the control signal used to generate sound.
Passive Buzzer	GND	Connects to GND on the MaaZEDU Development Board.

Table 7.3.2. Passive Buzzer connection for Lab 07

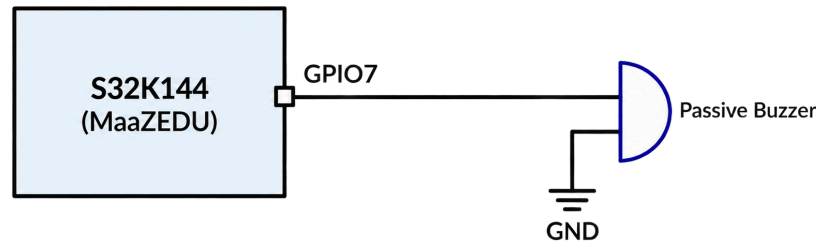


Figure 7.3.1. Passive Buzzer connection to GPIO7

4 EduFramework API

The Arduino Language Reference uses `tone()` and `noTone()` for tone generation and stopping operations [4]. EduFramework provides APIs with the same names and an interface suitable for the MaaZEDU platform.

4.1 `tone()`

`tone()` starts generating a square wave at the specified frequency on a PWM-capable Logical Pin. In the current implementation, the signal is generated with a 50% duty cycle [6].

`tone`

Starts a tone at the specified frequency on a PWM Logical Pin.

Syntax

```
tone(pin, frequency);
```

Parameters

Parameter	Type / accepted value	Description
<code>pin</code>	PWM Logical Pin	Pin used to control the buzzer, for example <code>GPIO7</code> .
<code>frequency</code>	Frequency (Hz)	Frequency of the tone to be generated, measured in hertz.

Return value

No return value.

Example:

```
tone(GPIO7, 440U);
```

In EduFramework, `tone()` does not include a duration parameter. The signal continues until its frequency is changed by another call to `tone()` or it is stopped by `noTone()` [6]. Therefore, the duration of each tone in this lab is determined by placing `delay()` after the `tone()` call.

`tone()` uses the framework PWM layer to prepare the corresponding output, so the program does not need to call `pinMode(GPIO7, OUTPUT)` before generating a tone [6].

4.2 noTone()

`noTone()` stops the signal currently being generated on the specified Logical Pin [6] .

noTone

Stops the tone on the specified PWM Logical Pin.

Syntax

```
noTone(pin);
```

Parameters

Parameter	Type / accepted value	Description
<code>pin</code>	PWM Logical Pin	Pin currently being used to generate a tone.

Return value

No return value.

Example:

```
noTone(GPIO7);
```

In this exercise, the API is called after the final tone to create a silent interval before the sequence repeats.

4.3 PWM Resource Note

Channels within the same FTM of the S32K1xx share a common 16-bit counter [5] . Because the output frequency depends on this shared counter, functions that require different frequencies should not be used simultaneously on pins belonging to the same FTM.

According to the current EduFramework PWM mapping, the Logical Pins are divided into the following groups [6] :

PWM Group	Logical Pin
Group 1	LED_RED , LED_BLUE , LED_GREEN
Group 2	GPIO7 , GPIO8
Group 3	GPIO2 , GPIO3 , GPIO4 , GPIO5 , GPIO6

Table 7.4.1. Logical Pin groups that share PWM resources

Note

Do not use `tone()` and `analogWrite()` simultaneously on two pins that belong to the same PWM group. For example, `GPIO7` and `GPIO8` both belong to Group 2. EduFramework currently does not implement runtime resource locking, so the application is responsible for selecting appropriate pins.

5 Lab Exercise

5.1 Requirements

Build a program that uses a passive buzzer on `GPIO7` to play a sequence of tones with increasing pitch. The program must satisfy the following requirements:

- Play seven tones corresponding to C, D, E, F, G, A, and B in sequence.
- Maintain each tone for 500 ms .
- After the final tone, stop the buzzer for 1000 ms .
- Repeat the entire sequence continuously.

5.2 Program

In `src/main.c` , implement the program that has been verified on the hardware:

```
#include "Arduino.h"

int main(void)
{
    setup();

    while (1)
    {
        tone(GPIO7, 262U); /* C */
        delay(500U);

        tone(GPIO7, 294U); /* D */
        delay(500U);

        tone(GPIO7, 330U); /* E */
        delay(500U);

        tone(GPIO7, 349U); /* F */
        delay(500U);

        tone(GPIO7, 392U); /* G */
        delay(500U);

        tone(GPIO7, 440U); /* A */
        delay(500U);

        tone(GPIO7, 494U); /* B */
        delay(500U);

        noTone(GPIO7);
        delay(1000U);
    }

    return 0;
}
```

Listing 7.5.1. Program for playing a tone sequence with a Passive Buzzer

Each pair of statements contains one frequency value and a 500 ms delay. As the program changes from the current frequency to a higher value, the perceived pitch increases according to the relationship between frequency and pitch [1] .

After 494 Hz , the program stops the buzzer signal and waits for 1000 ms . This silent interval creates a clear boundary between two repetitions of the sequence. When the loop starts again, the buzzer resumes from the first frequency.

The frequency values are written directly in the program so that the exercise focuses on the effect of the frequency parameter. A method for naming these frequencies to improve source-code readability is introduced in the Extension section.

5.3 Verification

Build and upload the program to the MaaZEDU Development Board. Listen to the sound produced by the passive buzzer and verify that:

- Seven consecutive tones are produced in each sequence.
- The pitch increases from the first tone to the last tone.
- Each tone lasts approximately 500 ms.
- A silent interval of approximately 1000 ms occurs after the final tone.
- The sequence repeats continuously.

EXPECTED RESULT

The passive buzzer produces seven tones with increasing pitch. Each tone is maintained for approximately 500 ms. After the final tone, the buzzer remains silent for approximately 1000 ms, then the sequence starts again.

6 Extension

When a program uses many tones, writing raw frequency values at every location can make the source code more difficult to read. A simple approach is to assign names to commonly used values with constants.

6.1 Naming Frequencies with Constants

For example, the first frequency used in the exercise can be defined as:

```
#define NOTE_C4 262U
```

The API call can then be written as:

```
tone(GPI07, NOTE_C4);
```

The constant name expresses the meaning of the value in the program without changing the frequency passed to the API.

6.2 Extension Exercise: Create a Short Melody

Replace the ascending sequence in the main exercise with a short melody of your own design.

Requirements:

- Use at least four different frequencies.
- Use at least two different tone durations.
- Include at least one silent interval between parts of the melody.
- Constants such as `NOTE_...` may be defined to improve source-code readability.
- Repeat the melody so that it can be verified multiple times on the hardware.

No fixed solution is provided for this extension. The frequencies, tone order, and durations are selected according to the melody being created.

EXPECTED RESULT

The passive buzzer produces a melody different from the sequence used in the main exercise, with noticeable changes in pitch, duration, and silent intervals.

7 References

- [1] University of Minnesota, *Pitch is Frequency*, Introduction to Sensation and Perception.
- [2] University of Southern California, *EE 109 Unit H - Timers*, EE 109 - Introduction to Embedded Systems.

- [3] Augusta University, *PHYS 1111L Laboratory Manual* , PHYS 1111L, 2023.
- [4] Arduino, *Arduino Language Reference* , Arduino Documentation.
- [5] NXP Semiconductors, *S32K1xx Series Cookbook* , AN5413, Rev. 5, 2020.
- [6] EduFramework, *Tone, PWM and Logical Pin API* , EduFramework Source Code.
- [7] MaaZEDU, *MaaZEDU Development Board Guide* .